

J1939 Fleet Telemetry Pipeline

MF4 → DBC Decode → Parquet → S3 → Grafana

Jayandhan Senthil Murugan

Mentor Summary

This project implements a two-step data pipeline for J1939 heavy vehicle telemetry: raw CAN bus recordings (MF4 format) are decoded against a J1939 DBC file and stored as columnar Parquet files in Amazon S3, then queried and visualized in Grafana via Amazon Athena. The pipeline was built and validated end-to-end using CSS Electronics open-source tooling, real J1939 sample data, and AWS cloud infrastructure.

What Was Built

Step	What it does
Step 1	Pull MF4 file from S3 input bucket → decode using J1939 DBC → write Parquet files to S3 output bucket, organized by device ID / signal group / date.
Step 2	AWS Glue crawls the Parquet output bucket → catalogs tables in Athena → Grafana queries Athena and renders live time-series dashboards.

Tech Stack

Layer	Tool / Service	Purpose
Raw data format	MF4 (MDF4)	Binary CAN bus log format from CANedge hardware
Signal decoding	J1939 DBC + mdf2parquet_decode.exe	Translates raw CAN messages to named J1939 signals
Output format	Apache Parquet	Columnar format — efficient for time-series analytics queries
Cloud storage	Amazon S3	Input bucket (MF4) + output bucket (Parquet)
Data catalog	AWS Glue Crawler	Scans Parquet output and registers tables in Glue catalog
Query engine	Amazon Athena	Serverless SQL queries directly on S3 Parquet files
Visualization	Grafana (Docker) + Athena plugin	Live time-series dashboards from Athena queries
Orchestration	CSS Electronics canedge-mdftoparquet-automation	Open-source pipeline repo connecting all steps
Local environment	Python 3 + venv + boto3	Local test runner triggering the Lambda-equivalent logic

AWS Infrastructure

Resource	Name	Role
S3 bucket (input)	canedge-mf4-input	Stores raw MF4 files + DBC file
S3 bucket (output)	canedge-mf4-input-parquet	Stores decoded Parquet files
S3 bucket (Athena)	canedge-athena-results	Stores Athena query result files
Glue database	canedge_db	Holds cataloged table definitions
Glue crawler	canedge-crawler	Auto-discovers Parquet schema in S3
Glue IAM role	AWSGlueServiceRole-canedge	Grants Glue access to S3 and catalog
IAM user	can-project1	Programmatic access for pipeline + Grafana

S3 Output Structure

Each MF4 file decoded by the pipeline produces Parquet files organized by device ID, signal group, and date:

```
canedge-mf4-input-parquet/  
  958D2219/                               ← device serial number  
    CAN1_EEC1_S00/                         ← J1939 signal group (engine speed)  
      2020/01/13/                          ← recording date  
        00002501_00002081.parquet  
    CAN1_CCVS1_S00/                       ← J1939 signal group (vehicle speed)  
      2020/01/13/  
        00002501_00002081.parquet
```

Athena Tables

Two J1939 signal groups were decoded from the sample data and are queryable in Athena:

Table	J1939 Message	Key signals
can1_eec1_s00	EEC1	enginespeed (RPM) — engine controller #1
can1_ccvs1_s00	CCVS1	wheelbasedvehiclespeed (km/h) — cruise control / vehicle speed

Grafana Dashboard Queries

Engine Speed (RPM)

```
SELECT t, enginespeed FROM can1_eec1_s00 ORDER BY t ASC
```

Vehicle Speed

```
SELECT t, wheelbasedvehiclespeed FROM can1_ccvs1_s00 ORDER BY t ASC
```

Visualization type: Time series. Timestamp column: t. Both panels show 13 Jan 2020 08:47–09:00 UTC — approximately 13 minutes of real truck telemetry from a single CANedge device.

How to Reproduce

1. Clone repo:

```
git clone https://github.com/CSS-Electronics/canedge-mdftoparquet-automation.git
```

2. Install dependencies and configure AWS:

```
pip install -r requirements.txt | aws configure
```

3. Upload MF4 files and DBC to S3 input bucket, then run the decoder:

```
python run_test.py --cloud Amazon --input-bucket canedge-mf4-input --object-path  
AABBCCDD/00000001/FILE.MF4
```

4. Run the Glue crawler to catalog Parquet output:

```
aws glue start-crawler --name canedge-crawler --region us-east-1
```

5. Start Grafana and query via Athena:

```
docker run -d --name grafana -p 3000:3000 -e AWS_ACCESS_KEY_ID=... -e  
AWS_SECRET_ACCESS_KEY=... -e AWS_DEFAULT_REGION=us-east-1 grafana/grafana-enterprise
```

Key Observations

- 145,534 raw CAN messages per MF4 file decode to ~21,542 named J1939 signals — a ~7x reduction via DBC filtering.
- The pipeline auto-organizes output by device ID, signal group, and date — no manual partitioning needed.
- Athena enables serverless SQL on S3 Parquet with no database to manage — query cost scales with data scanned, not uptime.
- Engine speed (EEC1) and vehicle speed (CCVS1) are the two most diagnostically useful J1939 signals for fleet PdM — correlated anomalies between these two signals are a primary fault indicator, consistent with the multi-signal reasoning in the companion analysis document.